The AI Financial Dashboard

Repo Link:

https://github.com/Omjnr06/ai-financial-dashboard/

1. Project Overview & Vision

Traditional banking apps and budgeting tools are fundamentally broken for young adults. They are static, backward-looking, and act like nagging calculators—telling you that you overspent only *after* the money is gone.

This project is a highly secure, multi-user personal finance platform that flips that dynamic. By combining automated banking data, machine learning, and a highly customizable UI, we are building a proactive "financial co-pilot." It doesn't just track pennies; it learns your hidden habits, predicts your future cash flow, and tells you exactly what is safe to spend *today*.

Beyond solving our own budgeting struggles, this platform is designed as a heavy-hitting, full-stack portfolio piece that demonstrates production-level data engineering, API integration, and machine learning. Our goal is to have an MVP deployed before the Fall term begins at Western.

2. The User POV (The Experience)

Imagine opening a banking app that actually feels like it belongs to you.

Zero Data Entry: You log in, authenticate your bank via Plaid once, and you are done. You never have to manually log a coffee purchase or categorize a grocery run again. **The "Safe to Spend" Number:** Instead of opening the app to see a scary "Checking Account Balance" (which is a trap because rent is due on Friday), the first thing you see is a massive, clean number: Your *Safe to Spend* amount. The app has already done the math, subtracting your upcoming bills and savings goals. **Instant Gratification (Power User Mode):** You are at the store. You tap your iPhone using Apple Pay. Before you even walk out the door, your Apple Shortcut pings the app's webhook, and your dashboard updates instantly. **Guiding, Not Nagging:** You randomly drop $100 on Uber Eats. The app doesn't yell at you. Instead, the anomaly detection engine flags it, while the savings predictor dynamically updates, showing that your "New MacBook" goal has

been pushed back by 3 days. It gives you the agency to make your own choices based on real data.

3. The "Vibe" & Ultimate Customizability

Personal finance is highly personal, so the UI must reflect the user. We are ditching the boring corporate blue of traditional banks.

- **The Customization Engine:** Users have complete control over their dashboard's aesthetic.
- **Color & Gradients:** Users aren't locked into "Light Mode" or "Dark Mode." They can select custom hex codes for their primary accents. They can apply dynamic background gradients (e.g., "Midnight Neon," "Cyberpunk Pink," or "Minimalist Monochrome").
- **Data Visualization Themes:** The Recharts visualizations will adapt to the user's chosen color palette. If they select a "Neon Glow" theme, the line charts forecasting their savings will glow with SVG drop-shadows.
- **Modular Dashboard:** Users can toggle what they see. Don't care about the Habit Clusters today? Hide it. Want the "Safe to Spend" number front and center? Pin it.

4. How It Works (Under the Hood)

To achieve this seamless experience, the app relies on a modern 5-layer architecture:

1. **Ingestion:** Plaid constantly monitors the user's bank. When a transaction happens, Plaid fires a secure webhook to our backend. For power users, an Apple Pay shortcut acts as a secondary, instant trigger.
2. **Deduplication:** Our Python backend catches these webhooks and intelligently merges them so temporary pending transactions are seamlessly replaced by official bank records without double-counting.
3. **Machine Learning:** Raw transactions are fed into our Scikit-Learn models.
    a. *K-Means Clustering* looks for hidden spending habits (e.g., grouping late-night fast food).
    b. *Isolation Forests* look for sudden statistical spikes to flag anomalies.
    c. *Time-Series Regression* looks at past spending variance to predict exactly what day a user will hit a savings goal.
4. **Storage:** Everything is securely isolated in a PostgreSQL database. We use **Better Auth** so that the users table lives directly in our database. This allows for strict Row-Level Security and complex SQL joins for our ML models.

5. **Delivery:** The Next.js frontend fetches these insights and renders them in a blazing-fast, customized, animated UI.

5. Team Organization & Roles

- **Backend & ML Architect (Lead):** Focuses on the FastAPI Python backend, the PostgreSQL schema, the data deduplication logic, and building the Scikit-Learn predictive models. Mentors the team on API integration.
- **Data & Auth Integration (Lead):** Focuses on implementing Better Auth for secure login and building the Next.js Plaid integration to successfully pull bank data and pass it to the backend.
- **Frontend & Visualization (Lead):** Focuses on the Next.js dashboard architecture, implementing the Recharts data visualizations, and building the Tailwind/Aceternity customization engine (themes, colors, layout).

6. Core Features Summary

- **Automated Data Ingestion:** Plaid + optional Apple Pay webhooks.
- **Goal-Based Savings Predictor:** ML-driven probabilistic tracking for custom savings buckets.
- **Hidden Habit Clustering:** Unsupervised ML that finds patterns in user spending.
- **Financial Anomaly Detection:** Automated alerts for statistical spending spikes.
- **The "Safe to Spend" Metric:** Real-time disposable cash calculation.
- **Manual Income/Outflow Entries:** Ability to manually add freelance cash or one-off purchases to keep the ML models perfectly accurate.

7. The Final Tech Stack

Component	Technology Selected	Reason for Choice
Frontend Framework	Next.js (TypeScript)	Industry standard; robust routing and React Server Components.

Frontend Styling	Tailwind CSS + Aceternity UI	Maximum flexibility for our custom themes, gradients, and animations.
Data Visualization	Recharts	Low-level SVG control for fully theme-able, glowing, and custom charts.
Authentication	Better Auth	Open-source, free. Keeps our User tables in *our* database, which is critical for complex Machine Learning SQL joins.
Backend API	FastAPI (Python)	High performance; natively speaks the same language as our ML scripts.
Database Engine	Neon (Serverless PostgreSQL)	Relational power needed for complex financial ledgers and user isolation.
Database ORM	SQLModel	Eliminates code duplication by unifying Pydantic schemas and DB models.
Machine Learning	Scikit-Learn / Keras	Handles Clustering, Anomaly Detection, and Savings Trajectory math.
Infrastructure	Docker	Containerizes the backend so the team can build without environment bugs.
Hosting	Vercel (Front) / Render (Back)	Seamless deployment directly from GitHub.

Workflow:

Phase 1: The Foundation (Weeks 1-2)

Before any fun charts or AI models are built, the core infrastructure must be locked down.

1. **Database Schema & Provisioning:**
    a. Spin up the Neon PostgreSQL database.
    b. Write the SQL/SQLModel scripts to create the exact tables: users, accounts, transactions, and savings_goals.
2. **Authentication (Better Auth):**
    a. Install Better Auth in the Next.js client.
    b. Ensure that when a user signs up, their secure UUID is correctly written to your new PostgreSQL users table. *Nothing else can happen until a user exists in the database.*
3. **The API Bridge:**
    a. Build a simple "Hello World" endpoint in the FastAPI server.
    b. Have the Next.js client successfully fetch that data. This proves your client and server can communicate.

Phase 2: Data Ingestion Pipeline (Weeks 2-3)

This is where the app comes alive. You need to get real bank data flowing into your database as fast as possible so the ML models have something to train on later.

1. **Plaid Link Integration (Client):**
    a. Build the UI button that opens the Plaid pop-up so the user can log into their bank.
2. **Plaid Webhook Receiver (Server):**
    a. Build the FastAPI endpoints that listen for Plaid's transaction updates (/api/plaid/webhook).
    b. Write the logic to parse Plaid's JSON payload and insert those rows into your transactions table, linking them to the correct user_id.
3. **The Apple Pay Shortcut Endpoint:**
    a. Build the secondary FastAPI endpoint (/api/transactions/instant) that accepts simple POST requests from an iPhone shortcut.
4. **The Deduplication Logic:**

    a. Write the database queries that check for matching amounts and dates, ensuring the Plaid webhooks merge with the Apple Pay temporary data instead of duplicating it.

Phase 3: Core UI & Visualization (Weeks 4-5)

Now that you have real financial data sitting in the database, you can build the dashboard to visualize it.

1. **The Customization Engine:**
    a. Set up Tailwind CSS, Aceternity UI, and the global state (like React Context or Zustand) to manage the user's color themes and dark/light modes.
2. **Standard Data Fetching:**
    a. Write the FastAPI routes to send basic data to the client (e.g., "Get last 10 transactions," "Get current account balance").
3. **Recharts Implementation:**
    a. Build the foundational graphs. Start with the "Income vs. Burn Rate" doughnut chart and the basic historical spending bar chart. Ensure these charts dynamically change color based on the Customization Engine.
4. **The "Safe to Spend" Widget:**
    a. Write the standard SQL aggregations to calculate current balance minus upcoming fixed bills, and display it as the hero component.

Phase 4: Machine Learning & Intelligence (Weeks 5-6)

With a working app and flowing data, you can finally add the "AI Co-Pilot" intelligence.

1. **Feature Engineering (Pandas):**
    a. Write the Python scripts that pull raw rows from PostgreSQL and clean them into structured dataframes (calculating variances, standard deviations, and rolling averages).
2. **Financial Anomaly Detection:**
    a. Implement the Isolation Forest model using Scikit-Learn.
    b. Create the logic that flags a transaction as an ANOMALY immediately as it enters the database, triggering an alert on the UI.
3. **Hidden Habit Clustering:**
    a. Implement the K-Means algorithm to group transactions by time-of-day and amount. Build the Recharts scatter plot to visualize these clusters on the client.
4. **The Savings Predictor:**

    a. Build the time-series forecasting model to calculate the dynamic "MacBook Pro" completion date. Build the Recharts probability cone to display the optimistic, likely, and pessimistic dates.

Phase 5: Polish & Deployment (Week 7)

Getting the application out of your local machines and onto the live internet.

1. **Containerization:**
    a. Write the Dockerfile for the FastAPI server to ensure it runs flawlessly in the cloud.
2. **Cloud Deployment:**
    a. Push the Next.js client to Vercel.
    b. Push the Dockerized FastAPI server to Render or Railway.
3. **Environment Variable Audit:**
    a. Ensure all Plaid API keys, Better Auth secrets, and Database URLs are securely stored in the production environments and not hardcoded.

Pages to be Made:

Priority Pages

- Landing Page / Home
- Sign Up / Register
- Log In
- Onboarding: Connect Bank (Plaid Link)
- Main Dashboard (Overview & "Safe to Spend")
- Transactions Ledger
- Add/Edit Manual Transaction
- Analytics & Insights (Habit Clusters, Cash-Flow Forecaster, Burn Rate)
- Savings Goals Dashboard
- Create/Edit Savings Goal
- Settings: Theming & Customization (Color/Gradient Builder)
- Settings: Integrations & API (Apple Pay Webhook setup, Plaid connections)
- Settings: Account & Profile

Edge Cases & System States

- 404 Not Found
- 500 Internal Server Error / Global Error Boundary

- Forgot Password / Reset Password
- Loading / Syncing Data State
- Empty States (Dashboard with 0 transactions, No Savings Goals created)
- Bank Connection Failed / Re-authentication Required
- Anomaly Alert Detail / Notification View
- Account Deletion Confirmation